

Univezitet u Kragujevcu,
Prirodno-matematički fakultet

Kompresija i zaštita podataka

Projektni zadatak 1

Student: Veljko Đurović, 41/2021

Mentor: dr Marko Petković, red. prof.

Kragujevac, 2026.

1. Uvod

Cilj projektnog zadatka je implementacija i poređenje osnovnih metoda za kompresiju podataka. Program je realizovan u programskom jeziku C# i omogućava izračunavanje bajt-entropije ulaznog binarnog fajla, kompresiju i dekompresiju pomoću Shannon–Fano, Huffman, LZ77 i LZW algoritama, kao i računanje ostvarenog stepena kompresije.

Ulazni alfabet čine sve moguće vrednosti jednog bajta, odnosno skup $A = \{0, 1, \dots, 255\}$. Program radi direktno nad nizovima bajtova, pa se isti postupak može primeniti na tekstualne i binarne fajlove. Kod algoritama koji proizvode niz bitova, podaci se pakuju u bajtove kako bi se svih osam bitova svakog izlaznog bajta iskoristilo.

Za svaki algoritam implementirane su funkcije za kompresiju i dekompresiju. Program kreira odvojene direktorijume za kompresovane i dekompresovane fajlove, a dobijeni rezultati mogu se porediti sa originalnim fajlom radi provere ispravnosti.

2. Organizacija programa

Program je organizovan tako da svi algoritmi za kompresiju implementiraju zajednički interfejs *ICompressionInterface*. Interfejs definiše metode Compress i Decompress, koje primaju i vraćaju niz bajtova, kao i svojstvo Name za naziv algoritma. Ovakva organizacija omogućava da se različiti algoritmi koriste preko istog tipa promenljive.

Struktura projekta:

- Algorithms – klase ShannonFano, Huffman, LZ77 i LZW
- Interfaces – interfejs ICompressionInterface
- Utils – pomoćne klase AlgorithmUtils i FileUtils
- Program.cs – učitavanje ulaza, pokretanje svih algoritama i prikaz rezultata

Pomoćna klasa AlgorithmUtils sadrži računanje frekvencija i entropije, računanje ukupnog broja kodiranih bitova, pakovanje kodova u bajtove i metode za upisivanje i čitanje proizvoljnog broja bitova. Klasa FileUtils služi za čitanje, upisivanje i poređenje fajlova, kao i računanje procenta uštede prostora.

3. Shannon–Fano algoritam

3.1. Princip rada

Najpre se računaju frekvencije svih bajtova koji se pojavljuju u fajlu. Za svaki takav bajt formira se čvor, a čvorovi se sortiraju opadajuće prema frekvenciji. Zatim se skup čvorova deli na dve uzastopne grupe tako da razlika suma njihovih frekvencija bude što manja. Simbolima u levoj

grupi dodaje se bit 0, a simbolima u desnoj grupi bit 1. Postupak se rekurzivno ponavlja dok svaka grupa ne sadrži samo jedan simbol.

3.2. Struktura kompresovanog fajla

U implementaciji se kod ne čuva kao niz tekstualnih nula i jedinica. Umesto toga čuvaju se frekvencije, na osnovu kojih se pri dekompresiji ponovo konstruiše potpuno isti Shannon–Fano kod. Format izlaznog fajla je:

- 1024 bajta – 256 frekvencija tipa Int32 (po 4 bajta)
- 4 bajta – ukupan broj validnih kodiranih bitova
- preostali bajtovi – kodirani podaci, pakovani po 8 bitova u bajt

Ukupna veličina zaglavlja iznosi 1028 bajtova. Broj validnih bitova je potreban zato što poslednji izlazni bajt može sadržati neiskorišćene dopunske bitove.

3.3. Dekompresija

Pri dekompresiji se najpre čita tabela frekvencija i broj validnih bitova. Na osnovu frekvencija ponovo se formiraju i sortiraju čvorovi i primenjuje isti postupak podele. Zatim se kodirani podaci čitaju bit po bit. Kada trenutno pročitani niz bitova odgovara nekom kodu, u izlaz se dodaje odgovarajući bajt. Posebno je obrađen slučaj kada ulaz sadrži samo jednu različitu vrednost bajta.

4. Huffmanov algoritam

4.1. Princip rada

Za svaki bajt sa nenultom frekvencijom formira se list stabla. U svakom koraku biraju se dva čvora sa najmanjim frekvencijama, spajaju u novi roditeljski čvor čija je frekvencija zbir njihovih frekvencija i vraćaju u skup čvorova. Postupak se ponavlja dok ne ostane samo koren Huffmanovog stabla. Levoj grani dodeljuje se bit 0, a desnoj bit 1.

4.2. Struktura kompresovanog fajla

Struktura izlaznog fajla ista je kao kod Shannon–Fano algoritma: 256 Int32 frekvencija, zatim Int32 vrednost ukupnog broja validnih bitova i na kraju bitovski upakovani kompresovani podaci. Na ovaj način nije potrebno posebno čuvati celo stablo ili tekstualni zapis kodova, jer se stablo rekonstruiše iz frekvencija.

4.3. Dekompresija

Na osnovu sačuvanih frekvencija ponovo se gradi Huffmanovo stablo. Kodirani deo se čita od najvišeg ka najnižem bitu svakog bajta. Bit 0 vodi u levo podstablo, a bit 1 u desno. Kada se dostigne list, njegov bajt se dodaje izlazu i čitanje sledećeg simbola ponovo počinje od korena. Čitanje prestaje nakon tačno onoliko bitova koliko je zapisano u zaglavlju.

5. LZ77 algoritam

5.1. Princip rada

LZ77 koristi klizni prozor i prethodno obrađeni deo ulaza kao rečnik. Za svaku trenutnu poziciju pretražuje se najviše windowLength prethodnih bajtova i traži najduže poklapanje. Rezultat se predstavlja tokenom (offset, length, nextSymbol). Offset označava udaljenost početka poklapanja unazad, length broj kopiranih bajtova, a nextSymbol sledeći literalni bajt.

U glavnom programu koristi se prozor dužine 1024. Za ovu vrednost offset zauzima 11 bitova, jer se broj potrebnih bitova računa iz veličine prozora. Dužina poklapanja ograničena je na 255 i zapisuje se u 8 bitova, dok se sledeći simbol takođe zapisuje u 8 bitova.

5.2. Struktura kompresovanog fajla

- 4 bajta – originalna dužina fajla (Int32)
- 4 bajta – veličina kliznog prozora (Int32)
- niz tokena – offsetBits bitova za offset, 8 bitova za length i 8 bitova za nextSymbol

Tokeni nisu poravnati na granice bajtova, već se njihove vrednosti redom upisuju u zajednički bitovski tok. Na kraju se nepotpun poslednji bajt dopunjava nulama.

5.3. Dekompresija

Najpre se čitaju originalna dužina i veličina prozora. Iz veličine prozora ponovo se određuje broj bitova za offset. Za svaki token se, ako su offset i length veći od nule, kopira length bajtova sa pozicije output.Count – offset, a zatim se dodaje nextSymbol. Originalna dužina sprečava da dopunski bitovi ili završni simbol proizvedu višak bajtova.

6. LZW algoritam

6.1. Princip rada

Početni rečnik sadrži svih 256 pojedinačnih bajtova sa kodovima 0–255. Algoritam održava najduži niz w koji već postoji u rečniku. Ako se spajanjem w i sledećeg bajta c dobije niz koji je već poznat, pretraga se nastavlja sa tim dužim nizom. U suprotnom se u izlaz upisuje kod za w, novi niz wc dodaje se u rečnik, a w postaje c.

Rečnik koristi kodove tipa UInt16. Kodovi se ne zapisuju uvek sa 16 bitova: početna širina je 9 bitova, a zatim se povećava kako rečnik raste, sve do najviše 16 bitova. Time se za manje kodove izbegava nepotrebno zauzimanje prostora.

6.2. Struktura kompresovanog fajla i dekompresija

LZW izlaz nema posebno zaglavlje. Sastoji se isključivo od promenljivo širokih kodova pakovanih u bitovski tok. Dekoder počinje sa istim početnim rečnikom i istom početnom širinom koda, pa rečnik gradi istim redosledom kao kompresor. Obrađen je i poseban LZW slučaj kada

pročitani kod još nije u rečniku, ali je jednak sledećem kodu koji treba dodeliti; tada se trenutni niz formira kao prethodni niz spojen sa njegovim prvim bajtom.

7. Pakovanje bitova

Za optimalnije skladištenje kodirani podaci se ne čuvaju kao karakteri '0' i '1'. Metoda WriteBits uzima celobrojnu vrednost i zadati broj bitova, izdvaja bitove od najznačajnijeg ka najmanje značajnom i upisuje ih u tekući izlazni bajt. Kada se popuni osam bitova, bajt se dodaje izlazu. Metoda ReadBits obavlja obrnut postupak i omogućava čitanje proizvoljnog broja bitova iz bitovskog toka.

8. Pokretanje programa

Program se pokreće kao konzolna aplikacija. Postupak rada je sledeći:

1. Pokrenuti aplikaciju.
2. Uneti punu putanju do ulaznog fajla.
3. Uneti putanju do postojećeg izlaznog direktorijuma.
4. Program izračunava i prikazuje bajt-entropiju ulaznog fajla.
5. Redom se izvršavaju Shannon–Fano, Huffman, LZ77 sa prozorom 1024 i LZW kompresija i dekompresija.
6. U izlaznom direktorijumu automatski se kreiraju folderi Compressed i Decompressed.
7. Za svaki algoritam prikazuje se ostvarena procentualna ušteda prostora.

Ispravnost se proverava tako što se isti fajl najpre kompresuje, zatim dekompresuje, a dobijeni fajl poredi sa originalom bajt po bajt. U pomoćnoj klasi FileUtils postoji metoda CompareFiles koja koristi SequenceEqual. Poređenje se može izvršiti i spoljnim alatom, na primer komandom fc /b u Windows okruženju.

originalni fajl → kompresija → kompresovani fajl → dekompresija → poređenje

10. Rezultati testiranja

Kao što se može videti na slici ispod, za testiranje je korišćen fajl čija bajt-entropija iznosi 5,93 bitova po bajtu. Ovakva vrednost entropije ukazuje na to da fajl ima određeni potencijal za kompresiju, ali nije idealno kompresibilan.

Najbolji rezultat ostvario je LZW algoritam, koji je smanjio veličinu fajla za skoro 90%. Nakon kompresije izvršena je i dekompresija, a zatim su originalni i dekompresovani fajl upoređeni kako bi se proverilo da su identični, odnosno da tokom procesa kompresije i dekompresije nije došlo do gubitka podataka.

```
Entropija ulaznog fajla je: 5.93

[Shannon-Fano] Kompresija u toku...
[Shannon-Fano] Kompresija završena.
[Shannon-Fano] Dekompresija u toku...
[Shannon-Fano] Dekompresija završena.
[Shannon-Fano] Step kompresije: 25.29%
[Shannon-Fano] Ulazni fajl i dekompresovani fajl JESU identicni.

[Huffman] Kompresija u toku...
[Huffman] Kompresija završena.
[Huffman] Dekompresija u toku...
[Huffman] Dekompresija završena.
[Huffman] Step kompresije: 25.57%
[Huffman] Ulazni fajl i dekompresovani fajl JESU identicni.

[LZ77] Kompresija u toku...
[LZ77] Kompresija završena.
[LZ77] Dekompresija u toku...
[LZ77] Dekompresija završena.
[LZ77] Step kompresije: 76.31%
[LZ77] Ulazni fajl i dekompresovani fajl JESU identicni.

[LZW] Kompresija u toku...
[LZW] Kompresija završena.
[LZW] Dekompresija u toku...
[LZW] Dekompresija završena.
[LZW] Step kompresije: 86.32%
[LZW] Ulazni fajl i dekompresovani fajl JESU identicni.
```

Slika 1: Rezultat izvršavanja